

gemstone-rs Companion Manual

Setup, user manual, examples, cookbook, codegen, explorer, and
workbench in one PDF



Generated from the Markdown sources in `gemstone-rs/docs`.

Contents

gemstone-rs Documentation

- Start Here

- PDFs

- Shortest Path

Setup Guide

- What You Need

- Which Install Path Should I Use?

- Environment Variables

- First Login

- Development Checkout

User Manual

- Configuration

- Sessions

- Eval and Perform

- OOP and Value Conversion

- Globals

- Transactions

- Export-Set Lifetime

- Browser API

- Error Handling

Examples Guide

- Common Setup

- Example Map

- Suggested Learning Order

- Expected Output

- CLI Equivalents

- Codegen Workflow

- VS Code

- Later Examples

Cookbook

- Recipe 1: Open a Session and Evaluate Smalltalk

Recipe 2: Verify ``3 + 4``

Recipe 3: Write and Read ``UserGlobals``

Recipe 4: Commit a Write Safely

Recipe 5: Abort on Error

Recipe 6: Convert Rust Values to OOPs

Recipe 7: Fetch a String

Recipe 8: Send a Message and Keep the Raw OOP

Recipe 9: Send a Message and Marshal the Result

Recipe 10: Retain a Long-Lived OOP

Recipe 10A: Store a Rust Payload Under BridgeRoot

Recipe 11: Browse Dictionaries

Recipe 12: Browse a Class

Recipe 13: Diagnose a New Machine

Recipe 14: Inspect an OOP From the CLI

Recipe 15: Inspect BridgeRoot From the CLI

Recipe 16: Preview Codegen

Recipe 17: Check Generated Code in CI

Recipe 18: Generate Wrappers

Recipe 19: Discover a Config From a Live Stone

Recipe 20: Run the Local Explorer

Recipe 19: Enable Explorer Eval Explicitly

Recipe 20: Use the VS Code Workbench With a Checkout

Recipe 21: Verify Published Artifacts

gemstone-py vs gemstone-rs

Install Paths

Best Fit

Maturity Matrix

Feature Matrix

How They Should Work Together

Recommendation

BridgeRoot and Object Mapping

What Is Supported

Rust Example

- Typed Rust Struct Mapping
- Derive-Based Mapping
- Nested Read-Back
- Key Policy
- Transactions and Identity
- Comparison With MagLev/GBS
- Codegen Support
- Explorer and VS Code Support
- Current Boundaries

Rust Codegen

- Install
- Config Format
- Method Metadata
- Mapped Structs
- Commands
- Generated Shape
- Generated Wrapper App
- Generated Object Mapping
- VS Code Workflow

Explorer Guide

- Install and Run
- Safety Defaults
- Browse Endpoints
- Eval
- Codegen Endpoints
- BridgeRoot Endpoints
- Product Direction

VS Code Workbench

- Setup
- Sidebar Tree
- Command Palette
- Codegen Workflow
- Object Mapping Workflow

Embedded Explorer Webview	
Develop the Extension	
Later Feature Work	
Performance and Safety	
Dynamic GCI Loading	
Session Threading	
Benchmark Smoke Test	
Rust vs Python	
Safety Checklist	
Shared Core Integration	
What Should Move Into Rust	
What Should Stay Python-Specific	
PyO3 Wrapper Shape	
Migration Plan	
Release Checklist	
Before Release	
Publish	
Post-Release Verification	
Optional Live Smoke	

gemstone-rs Documentation

This directory contains the human-facing guides for `gemstone-rs`.

Start Here

Guide	Use it when
Setup Guide	You need Rust, GemStone environment variables, install commands, and first login checks.
Examples Guide	You want to know which example to run for each feature.
User Manual	You want the main Rust API surface in one place.
Cookbook	You want task-focused recipes for sessions, transactions, browser calls, codegen, explorer, and VS Code.
gemstone-py vs gemstone-rs	You want install paths, use cases, maturity, and feature differences between the Python and Rust projects.
BridgeRoot and Object Mapping	You want MagLev-style bridge-root storage and explicit Rust-to-GemStone value mapping.
Codegen Guide	You want generated Rust wrappers for GemStone classes and methods.
Explorer Guide	You want the local HTTP explorer API and safety defaults.
VS Code Workbench	You want sidebar browsing, codegen preview/diff/generate, and explorer launch commands.
Performance and Safety	You want benchmark guidance, GCI loading notes, and threading rules.
Shared Core Integration	You want the plan for <code>gemstone-py-native</code> to wrap the Rust core.
Medium Article	You want an article-style explanation suitable for publishing or sharing.
Funny Introduction	You want a lighter multi-part tour of the same concepts.
Release Checklist	You want the crate, VSIX, GitHub Release, and post-release verification flow.

PDFs

Generated PDFs live in **pdf/**. Rebuild them after Markdown changes:

```
python3 docs/build_pdf_docs.py
```

Shortest Path

```
cargo install gemstone-rs-cli  
gemstone-rs --help
```

For library use:

```
cargo add gemstone-rs
```

For local examples from a checkout:

```
cd /path/to/gemstone-rs  
cargo run -p gemstone-rs --example quickstart
```

For the local explorer:

```
cargo install gemstone-rs-explorer  
gemstone-rs-explorer --port 8787
```

For VS Code:

```
https://marketplace.visualstudio.com/items?itemName=unicompute.gemstone-rs-workbench
```

Setup Guide

This guide gets you from a Rust project or source checkout to a live GemStone login.

What You Need

At minimum:

- Rust stable toolchain
- access to a GemStone/S 64 stone
- GemStone client library access through `libgcirpc`
- GemStone username and password

For the full local development experience:

- a `gemstone-rs` checkout
- `cargo`, `rustfmt`, and `clippy`
- Node.js 22 when working on the VS Code extension
- a Marketplace PAT only when publishing the VSIX
- a crates.io API token only when publishing crates

Which Install Path Should I Use?

Use case	Command
Rust application dependency	<code>cargo add gemstone-rs</code>
CLI tools	<code>cargo install gemstone-rs-cli</code>
Local HTTP explorer	<code>cargo install gemstone-rs-explorer</code>
Source checkout examples	<code>cargo run -p gemstone-rs --example quickstart</code>
VS Code users	Install <code>unicompute.gemstone-rs-workbench</code> from Marketplace

Environment Variables

Most live commands use `Config::from_env()`:

```
export GS_LIB=/opt/gemstone/product/lib
export GS_STONE=gs64stone
export GS_STONE_NAME=gs64stone
export GS_USERNAME=DataCurator
export GS_PASSWORD=swordfish
```

`GS_STONE` is canonical. `GS_STONE_NAME` is accepted as an alias when `GS_STONE` is absent. Setting both to the same value keeps CLI, examples, explorer, and VS Code settings aligned.

Optional variables:

```
export GS_HOST=localhost
export GS_NETLDI=netldi
export GS_GEM_SERVICE=gemnetobject
export GS_HOST_USERNAME=
export GS_HOST_PASSWORD=
export GS_LIB_PATH=/full/path/to/libgcirpc.dylib
```

Variable	Required	Purpose
<code>GS_LIB</code>	Usually	GemStone <code>lib/</code> directory for runtime library discovery.
<code>GS_LIB_PATH</code>	No	Full path to a specific <code>libgcirpc</code> file.
<code>GS_STONE</code>	Yes	Stone name.
<code>GS_STONE_NAME</code>	No	Stone-name alias used when <code>GS_STONE</code> is absent.
<code>GS_USERNAME</code>	Yes	GemStone login username.
<code>GS_PASSWORD</code>	Yes	GemStone login password.
<code>GS_HOST</code>	No	Remote host, defaults to local behavior.
<code>GS_NETLDI</code>	No	NetLDI service name.
<code>GS_GEM_SERVICE</code>	No	Gem service name.

First Login

With the CLI:

```
cargo install gemstone-rs-cli
gemstone-rs doctor
gemstone-rs doctor --live
gemstone-rs doctor --json
gemstone-rs eval "3 + 4"
```

Expected output:

```
7
```

`gemstone-rs doctor` prints the relevant `GS_*` values with secrets masked, loads the configured `libgcirpc`, and reports setup problems without exposing passwords. Add `--live` when the stone should be reachable; it logs in and asserts that `3 + 4` returns `7`. Add `--json` when a script, CI job, or editor integration needs a parseable report.

From a source checkout:

```
cd /path/to/gemstone-rs
cargo run -p gemstone-rs --example hello_gemstone
cargo run -p gemstone-rs --example quickstart
```

From a Rust application:

```
use gemstone_rs::{Config, Session, Value};

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env())?;
    assert_eq!(session.eval("3 + 4")?, Value::SmallInt(7));
    Ok(())
}
```

Development Checkout

```
git clone git@github.com:unicompute/gemstone-rs.git
cd gemstone-rs
make verify
```

`make verify` runs formatting, `cargo check`, clippy, tests, codegen freshness, the VS Code extension syntax/smoke checks, and PDF generation.

Package the VSIX locally:

```
make vscode-package
```

Verify already-published artifacts:

```
scripts/publish_verify.sh 0.2.0
```

User Manual

`gemstone-rs` is the safe Rust client API for GemStone/S over GCI. Rust code imports it as `gemstone_rs`.

Configuration

Use `Config::from_env()` for normal applications:

```
let config = gemstone_rs::Config::from_env()?;
```

Use the builder when you want explicit configuration:

```
use gemstone_rs::Config;

let config = Config::builder()
    .stone("gs64stone")
    .host("localhost")
    .netldi("netldi")
    .username("DataCurator")
    .password("swordfish")
    .build()?;
```

Sessions

`Session` logs out automatically on drop. Calling `logout()` explicitly is still useful in examples and command-line tools.

```
use gemstone_rs::{Config, Session, Value};

let mut session = Session::login(Config::from_env())?;
let value = session.eval("3 + 4")?;
assert_eq!(value, Value::SmallInt(7));
session.logout()?;
```

`Session` is deliberately not `Send` or `Sync`. Keep a session on the thread that logged it in.

Eval and Perform

`eval` returns a marshalled `Value`:

```
let value = session.eval("3 + 4")?;
```

`execute` returns a raw `Obj`:

```
let object_class = session.execute("Object")?;
```

`perform` returns a marshalled `Value`:

```
let seven = session.smallint_oop(7);  
let printed = session.perform(seven, "printString", &[])?;
```

`perform_oop` returns a raw `Obj`:

```
let printed_oop = session.perform_oop(seven, "printString", &[])?;  
let printed = session.fetch_string(printed_oop)?;
```

OOP and Value Conversion

`Value` covers nil, booleans, small integers, characters, fetched strings, and raw OOPs:

```
use gemstone_rs::Value;  
  
let oop = session.value_to_oop(&Value::SmallInt(7))?;  
let value = session.eval("true")?;  
println!("{value:?}");
```

Helpers:

```
let nil = session.nil_oop();  
let yes = session.bool_oop(true);  
let seven = session.smallint_oop(7);  
let text = session.new_string("hello")?;  
let symbol = session.new_symbol("ExampleSymbol")?;
```

Globals

`global_get` and `global_put` use `UserGlobals`:

```
let text = session.new_string("hello from Rust")?;  
session.global_put("GemStoneRsText", text)?;
```

```
let stored = session.global_get("GemStoneRsText"?);
println!("{}", session.fetch_string(stored)?);
```

Transactions

Use `transaction` when you want commit-on-success and abort-on-error:

```
session.transaction(|session| {
    let value = session.new_string("committed"?);
    session.global_put("GemStoneRsCommitted", value)
})?;
```

Manual transaction primitives are also available:

```
let needs_commit = session.needs_commit()?;
let in_transaction = session.in_transaction()?;
session.commit()?;
session.abort()?;
```

Export-Set Lifetime

Use `retain_oop` when a raw OOP must be held across calls and protected from GemStone-side collection:

```
let text = session.new_string("retained"?);
let handle = session.retain_oop(text)?;
println!("{}", handle.oop().raw());
handle.release()?;
```

The handle releases on drop if you do not call `release()`.

Browser API

```
use gemstone_rs::browser::{Browser, ALL_PROTOCOLS};

let mut browser = Browser::new(&mut session);
let dictionaries = browser.dictionaries()?;
let classes = browser.classes("UserGlobals"?);
let protocols = browser.protocols("Object", false, "")?;
let methods = browser.methods("Object", ALL_PROTOCOLS, false, "")?;
let source = browser.source("Object", "printString", false, "")?;
```

Pass an empty dictionary to resolve through the active user's symbol list.

Error Handling

Most APIs return `gemstone_rs::Result<T>`. Errors distinguish missing environment, missing config, GCI loading/calls, GemStone errors, illegal OOPs, unexpected typed codegen returns, and conversion issues.

```
match Config::from_env() {
    Ok(config) => println!("stone {}", config.stone),
    Err(err) => eprintln!("configuration error: {err}"),
}
```

The CLI exposes the same checks through `doctor`:

```
gemstone-rs doctor
gemstone-rs doctor --live
gemstone-rs doctor --json
```

The non-live form validates environment and GCI library loading. The live form also logs in and checks that `3 + 4` returns `7`. The JSON form is intended for automation and editor integrations.

Examples Guide

The `gemstone-rs` examples are split between runnable Cargo examples and the checked-in codegen sample under the repository-level `examples/` directory.

Common Setup

```
export GS_LIB=/opt/gemstone/product/lib
export GS_STONE=gs64stone
export GS_STONE_NAME=gs64stone
export GS_USERNAME=DataCurator
export GS_PASSWORD=swordfish
```

Run Cargo examples from the repository root:

```
cd /path/to/gemstone-rs
cargo run -p gemstone-rs --example quickstart
```

Example Map

Feature	Command or path	What it demonstrates
First login	<code>cargo run -p gemstone-rs -- example hello_gemstone</code>	Reads env config, logs in, prints a session id, and evaluates <code>3 + 4</code> .
Quickstart	<code>cargo run -p gemstone-rs -- example quickstart</code>	Eval, <code>global_put</code> , <code>global_get</code> , string fetch, cleanup.
Eval only	<code>cargo run -p gemstone-rs -- example eval</code>	Minimal <code>Session::eval</code> shape.
Browser API	<code>cargo run -p gemstone-rs -- example browser</code>	Dictionaries, protocols, methods, and source.
Live smoke cookbook	<code>cargo run -p gemstone-rs -- example live_smoke_cookbook</code>	Login, eval, global round-trip, perform, and transaction checks in one run.
Transactions	<code>cargo run -p gemstone-rs -- example transactions</code>	Commit-on-success and abort-on-error transaction wrapper.
OOP/value conversion	<code>cargo run -p gemstone-rs -- example oop_values</code>	<code>Value</code> , <code>Oop</code> , strings, symbols, and export-set retention.
BridgeRoot mapping	<code>cargo run -p gemstone-rs -- example bridge_root_mapping</code>	MagLev-style bridge-root storage with explicit <code>BridgeValue</code> mapping.
Derive mapping	<code>cargo run -p gemstone-rs -- example derive_mapping</code>	<code>#[derive(BridgeMapped)]</code> , symbol keys, nested structs, vectors, and BridgeRoot transactions.
Offline codegen	<code>cargo run -p gemstone-rs -- example codegen_preview</code>	Generates wrappers from the sample config without a live stone.
Codegen workflow	<code>cargo run -p gemstone-rs -- example codegen_workflow</code>	Writes config, previews, diffs, checks, generates, and verifies a clean diff.
Codegen discovery	<code>cargo run -p gemstone-rs -- example codegen_discover</code>	Connects to a live stone and discovers a starter config for <code>Object</code> .
Mapping discovery	<code>cargo run -p gemstone-rs -- example codegen_discover_mapping</code>	Connects to a live stone and proposes a <code>BridgeMapped</code> config.
Generated wrapper app	<code>cargo run -p gemstone-rs -- example generated_wrapper_app</code>	Uses checked-in generated wrappers to call <code>Object>>printString</code> .

Feature	Command or path	What it demonstrates
Generated mapping app	<code>cargo run -p gemstone-rs -- example generated_mapping_app</code>	Uses codegen-created <code>BridgeMapped</code> structs with <code>BridgeRoot</code> .
Codegen files	<code>examples/codegen/</code>	Config, generated wrappers, check/diff/generate workflow.
Explorer tooling	<code>examples/tooling/explorer.md</code>	Local explorer startup and endpoint checks.
VS Code tooling	<code>examples/tooling/vscode-workbench.md</code>	Sidebar browsing, codegen actions, and explorer launch.
CLI browser walkthrough	<code>examples/tooling/cli-browser-walkthrough.md</code>	Terminal-only browse workflow.
Axum service sketch	<code>examples/axum-service/README.md</code>	Recommended web-service shape without adding workspace dependencies.

Suggested Learning Order

1. `hello_gemstone`
2. `quickstart`
3. `browser`
4. `live_smoke_cookbook`
5. `oop_values`
6. `transactions`
7. `bridge_root_mapping`
8. `derive_mapping`
9. `codegen_preview`
10. `codegen_workflow`
11. `generated_wrapper_app`
12. `generated_mapping_app`
13. `codegen_discover`
14. `codegen_discover_mapping`
15. `examples/codegen/`
16. `examples/tooling/cli-browser-walkthrough.md`
17. `examples/tooling/explorer.md`
18. `examples/tooling/vscode-workbench.md`
19. `examples/axum-service/README.md`

Expected Output

Live examples need GemStone credentials and a reachable stone. If the environment is configured correctly, these are the important lines to look for:

```
$ cargo run -p gemstone-rs --example hello_gemstone
session id: <number>
3 + 4 => SmallInt(7)

$ cargo run -p gemstone-rs --example quickstart
GemStone eval ok: SmallInt(7)
GemStoneRsQuickstart: hello from gemstone-rs quickstart

$ cargo run -p gemstone-rs --example generated_wrapper_app
generated wrapper printString: 7

$ cargo run -p gemstone-rs --example live_smoke_cookbook
login ok: session <number>
eval ok: SmallInt(7)
global round-trip ok
perform ok: 7
transaction commit/abort ok

$ cargo run -p gemstone-rs --example bridge_root_mapping
bridge root: GemStoneRsBridgeRoot
MyTestDict OOP: <number>
loaded payload: BookingDraft { name: "Tariq", amount: 100, currency: "GBP" }

$ cargo run -p gemstone-rs --example derive_mapping
derived mapped payload: BookingDraft { amount: 100, customer: CustomerDraft { name:
"Tariq" }, tags: ["priority", "demo"] }

$ cargo run -p gemstone-rs --example generated_mapping_app
generated mapped payload: BookingDraft { name: "Tariq", amount: 100, currency: "GBP",
tags: ["priority", "demo"] }
```

Offline examples should run without GemStone:

```
$ cargo run -p gemstone-rs --example codegen_workflow
before generate: exists=false up_to_date=false diff_bytes=<number>
after generate: exists=true up_to_date=true
diff after generate: clean
```

CLI Equivalents

The CLI gives you the same live checks without compiling examples:

```

cargo install gemstone-rs-cli
gemstone-rs doctor
gemstone-rs doctor --live
gemstone-rs doctor --json
gemstone-rs eval "3 + 4"
gemstone-rs browse dictionaries
gemstone-rs browse classes UserGlobals
gemstone-rs browse protocols Object
gemstone-rs browse methods Object "-- all --"
gemstone-rs browse source Object printString
gemstone-rs inspect oop 20
gemstone-rs bridge root
gemstone-rs bridge keys
gemstone-rs bridge sample-config BookingDraft

```

Codegen Workflow

```

cargo run -p gemstone-rs-cli -- codegen init examples/codegen/demo.codegen
cargo run -p gemstone-rs-cli -- codegen preview examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen diff examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen check examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen generate examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen discover-mapping examples/codegen/
mapping.codegen BookingDraft Object

```

Generate a starter config from a live stone:

```

cargo run -p gemstone-rs-cli -- codegen discover examples/codegen/discovered.codegen
Object
cargo run -p gemstone-rs --example codegen_discover_mapping

```

Run the generated wrapper example after checking the generated file into the repository:

```

cargo run -p gemstone-rs --example generated_wrapper_app

```

VS Code

Install the workbench from:

```

https://marketplace.visualstudio.com/items?itemName=unicompute.gemstone-rs-workbench

```

Use the GemStone RS sidebar to browse dictionaries, classes, protocols, and methods. The same sidebar exposes Codegen Discover, Preview, Diff, Check, Generate, Generate Mapping Config, Preview BridgeRoot, List BridgeRoot Keys, Put BridgeRoot String, Remove BridgeRoot Key, Run Generated Mapping Example, and Open Docs actions.

Later Examples

These are useful, but should wait until the corresponding APIs are stable:

- a local explorer workflow with screenshots
- a full Axum or Actix workspace member wired into CI
- a richer class browser walkthrough with captured output

Cookbook

This cookbook is a collection of direct `gemstone-rs` recipes. Each recipe is small enough to copy into a Rust CLI, service, or maintenance tool.

Recipe 1: Open a Session and Evaluate Smalltalk

```
use gemstone_rs::{Config, Session};

let mut session = Session::login(Config::from_env())?;
println!("{}", session.eval("3 factorial"));
```

Use this when you need the smallest live sanity check.

Recipe 2: Verify $3 + 4$

```
use gemstone_rs::{Config, Session, Value};

let mut session = Session::login(Config::from_env())?;
assert_eq!(session.eval("3 + 4"), Value::SmallInt(7));
```

This is the test to keep in every live smoke lane.

Recipe 3: Write and Read UserGlobals

```
use gemstone_rs::{Config, Oop, Session};

let mut session = Session::login(Config::from_env())?;
let key = "GemStoneRsCookbook";
let value = session.new_string("stored from Rust");
session.global_put(key, value)?;

let stored = session.global_get(key)?;
println!("{}", session.fetch_string(stored)?);

session.global_put(key, Oop::NIL)?;
```

Recipe 4: Commit a Write Safely

```
session.transaction(|session| {
    let value = session.new_string("committed");
    session.global_put("GemStoneRsCommitted", value)
})?;
```

The closure commits only when it returns `Ok`.

Recipe 5: Abort on Error

```
use gemstone_rs::Error;

let result: gemstone_rs::Result<()> = session.transaction(|session| {
    let value = session.new_string("will abort");
    session.global_put("GemStoneRsAborted", value)?;
    Err(Error::IllegalOop {
        operation: "intentional abort",
    })
});

assert!(result.is_err());
```

Recipe 6: Convert Rust Values to OOPs

```
use gemstone_rs::Value;

let seven = session.value_to_oop(&Value::SmallInt(7))?;
```

```
let yes = session.bool_oop(true);
let nothing = session.nil_oop();
```

Recipe 7: Fetch a String

```
let string_oop = session.new_string("hello"?);
let text = session.fetch_string(string_oop)?;
assert_eq!(text, "hello");
```

Recipe 8: Send a Message and Keep the Raw OOP

```
let seven = session.smallint_oop(7);
let printed_oop = session.perform_oop(seven, "printString", &[])?;
println!("{}", session.fetch_string(printed_oop)?);
```

Recipe 9: Send a Message and Marshal the Result

```
let seven = session.smallint_oop(7);
let value = session.perform(seven, "class", &[])?;
println!("{}", value);
```

Recipe 10: Retain a Long-Lived OOP

```
let text = session.new_string("retained"?);
let handle = session.retain_oop(text)?;
println!("{}", handle.oop().raw());
handle.release()?;
```

The handle releases automatically on drop if `release()` is not called.

Recipe 10A: Store a Rust Payload Under BridgeRoot

```
use gemstone_rs::{BridgeValue, Config, Session};

let mut session = Session::login(Config::from_env())?;
let payload = BridgeValue::dictionary([
    ("name".to_string(), BridgeValue::from("Tariq")),
```

```

    ("amount".to_string(), BridgeValue::from(100_i64)),
    ("currency".to_string(), BridgeValue::from("GBP")),
  ]);

  let mut bridge_root = session.bridge_root()?;
  bridge_root.put("MyTestDict", payload)?;
  bridge_root.commit()?;

```

The default root is `UserGlobals` at: `#GemStoneRsBridgeRoot`.

Recipe 11: Browse Dictionaries

```

use gemstone_rs::browser::Browser;

let mut browser = Browser::new(&mut session);
for dictionary in browser.dictionaries()? {
    println!("{dictionary}");
}

```

Recipe 12: Browse a Class

```

use gemstone_rs::browser::{Browser, ALL_PROTOCOLS};

let mut browser = Browser::new(&mut session);
let protocols = browser.protocols("Object", false, "");?;
let methods = browser.methods("Object", ALL_PROTOCOLS, false, "");?;
let source = browser.source("Object", "printString", false, "");?;

```

Pass an empty dictionary to resolve through the active user's symbol list.

Recipe 13: Diagnose a New Machine

```

gemstone-rs doctor
gemstone-rs doctor --live
gemstone-rs doctor --json

```

The first command checks environment and GCI loading. The second command logs in and verifies that `3 + 4` returns `7`. The JSON form is useful in CI or editor tooling.

Recipe 14: Inspect an OOP From the CLI

```
gemstone-rs inspect oop 20
```

This prints the raw OOP, class OOP, and `printString`.

Recipe 15: Inspect BridgeRoot From the CLI

```
gemstone-rs bridge root
gemstone-rs bridge keys
gemstone-rs bridge get BookingDraft --symbol
gemstone-rs bridge inspect BookingDraft --symbol
gemstone-rs bridge put WorkbenchDraft "hello from Rust" --type String
gemstone-rs bridge remove WorkbenchDraft
```

Use this when object-mapping examples or generated mappings write payloads under `GemStoneRsBridgeRoot`. `bridge keys` is the quickest way to see what is currently stored before inspecting a specific payload. `bridge put` and `bridge remove` are useful live-write smoke tests because they commit one explicit BridgeRoot change at a time.

Recipe 16: Preview Codegen

```
gemstone-rs codegen preview examples/codegen/gemstone-rs.codegen
```

Use preview before writing generated files.

Recipe 17: Check Generated Code in CI

```
gemstone-rs codegen check examples/codegen/gemstone-rs.codegen
```

The repository runs the same check through `make verify`.

Recipe 18: Generate Wrappers

```
gemstone-rs codegen generate examples/codegen/gemstone-rs.codegen
```

Generated files are meant to be checked in.

Recipe 19: Discover a Config From a Live Stone

```
gemstone-rs codegen discover examples/codegen/discovered.codegen Object
```

Use this to bootstrap a config, then edit the generated mapping down to the API you actually want.

Recipe 20: Run the Local Explorer

```
gemstone-rs-explorer --port 8787
```

Open:

```
http://127.0.0.1:8787/
```

The explorer starts read-only and loopback-only.

Recipe 19: Enable Explorer Eval Explicitly

```
gemstone-rs-explorer --port 8787 --allow-eval  
curl -s 'http://127.0.0.1:8787/api/eval?source=3%20%2B%204'
```

Use this locally only.

Recipe 20: Use the VS Code Workbench With a Checkout

```
{  
  "gemstoneRs.checkoutPath": "/path/to/gemstone-rs",  
  "gemstoneRs.useCargo": true,  
  "gemstoneRs.codegenConfig": "examples/codegen/gemstone-rs.codegen"  
}
```

Then open the GemStone RS sidebar and use the Dictionaries, Codegen Config, and Explorer trees.

Recipe 21: Verify Published Artifacts

```
scripts/publish_verify.sh 0.2.0
```

The script checks crates.io versions, installs the CLI and explorer, runs `--help`, and checks the Marketplace version.

gemstone-py vs gemstone-rs

`gemstone-py` and `gemstone-rs` solve related problems at different maturity levels.

Use `gemstone-py` when you want the most complete Python experience today: Python sessions, async examples, web framework adapters, native acceleration, codegen, VS Code workbench integration, and a Python database explorer.

Use `gemstone-rs` when you want Rust services, CLIs, workers, or tooling to talk to GemStone/S directly without keeping Python in the process.

Install Paths

Goal	gemstone-py	gemstone-rs
Application library	<code>python -m pip install gemstone-py</code>	<code>cargo add gemstone-rs</code>
Native acceleration	<code>python -m pip install "gemstone-py[fast]"</code>	Built into the Rust GCI path once <code>libgcirpc</code> is available
Examples from source	<code>python -m pip install -e ".[examples]"</code>	<code>cargo run -p gemstone-rs -- example quickstart</code>
CLI tooling	Python modules and examples	<code>cargo install gemstone-rs-cli</code>
Local explorer	<code>python-gemstone-database-explorer</code>	<code>cargo install gemstone-rs-explorer</code>
VS Code	<code>gemstone-py</code> Workbench	<code>gemstone-rs</code> Workbench

Both projects use the same GemStone-style environment:

```
export GS_LIB=/opt/gemstone/product/lib
export GS_STONE=gs64stone
export GS_USERNAME=DataCurator
export GS_PASSWORD=your-password
```

`GS_STONE_NAME` is accepted as a stone-name alias where the clients support it. Use `GS_LIB_PATH` when you want to point directly at a specific `libgcirpc` file.

Best Fit

Use case	Better choice	Why
Python web app	gemstone-py	FastAPI and Litestar examples are already first-class.
Python scripts and notebooks	gemstone-py	Lower friction for Python teams.
Rust service or worker	gemstone-rs	Native Rust API, Rust errors, Rust ownership, Cargo distribution.
Rust CLI tooling	gemstone-rs	CLI and explorer can run without Python.
VS Code database/codegen workflow	gemstone-py today, gemstone-rs later	gemstone-py has the more mature explorer; gemstone-rs has the cleaner Rust backend direction.
Shared low-level bridge	gemstone-rs	The Rust API is the right place to isolate GCI safety and ownership.

Maturity Matrix

Capability	gemstone-py	gemstone-rs
Stable install path	Mature: PyPI and TestPyPI	Early: crates.io workflow and verification added
Native bridge	PyO3 extension path	Rust GCI crate and safe API
Sync API	Mature	Initial safe API
Async API	Mature enough for examples and tests	Not yet a core feature

Capability	gemstone-py	gemstone-rs
Web frameworks	FastAPI, Litestar, Django examples	Axum service sketch added; full app example still planned
Codegen	Python wrapper workflow	Rust wrapper workflow with preview/diff/check/generate
Browser API	Used by database explorer	CLI/explorer API for dictionaries/classes/methods/source
Local explorer	More mature Python app	Minimal Rust explorer proving the API
VS Code extension	More complete product flow	Thin command/workbench layer over Rust CLI and explorer
Live tests	Broader coverage	Growing smoke suite for login/eval/perform/globals/transactions/codegen
Release automation	More complete	Added crates/VSIX/GitHub verification path
Docs	Broader and polished	Catching up with guide, cookbook, article, comparison, PDFs

Feature Matrix

Feature	gemstone-py status	gemstone-rs status
Login/logout	Supported	Supported
<code>3 + 4 == 7</code> smoke	Supported	Supported
Global put/get	Supported	Supported
String round-trip	Supported	Supported
<code>perform</code> and <code>perform_oop</code>	Supported	Supported
Commit/abort	Supported	Supported
OOP handles/export set	Supported	Supported
Browser dictionaries/classes/protocols/methods/source	Supported	Supported through <code>Browser</code> , CLI, and explorer

Feature	gemstone-py status	gemstone-rs status
Generated wrappers	Supported	Supported for selected classes/methods
Codegen diff/check/generate	Supported	Supported
Live codegen discovery	Supported	Supported through CLI/API
FastAPI demo	Supported	Not applicable
Litestar demo	Supported	Not applicable
Axum/Actix demo	Not applicable	Axum sketch documented; full app example still planned
Database explorer	Mature Python app	Rust explorer has browser UI and local API; still less polished
VS Code sidebar workflow	More complete	Rust workbench has sidebar commands and embedded explorer webview

How They Should Work Together

The best long-term shape is not competition between the projects. It is a clean boundary:

- `gemstone-rs` owns the direct Rust/GCI interface and safe Rust API.
- `gemstone-py-native` can become a thin PyO3 layer over the Rust core.
- `gemstone-py` remains the Python API, examples, and Python web integration.
- The Python and Rust explorers can share concepts and eventually converge on

common codegen workflows.

That gives Python users the friendliest path and Rust users a direct path, while keeping unsafe GCI behavior isolated in one Rust layer.

Recommendation

For production Python projects, start with `gemstone-py`.

For Rust-native services, CLIs, workers, and tooling, start with `gemstone-rs`, but treat it as an early API that should be validated against a live stone in your environment.

For VS Code and visual codegen work, use the existing Python explorer as the product reference and keep moving `gemstone-rs-explorer` toward the same level of capability.

BridgeRoot and Object Mapping

`gemstone-rs` now has a first object-mapping layer over the lower-level `Oop` API.

This is intentionally smaller than MagLev/GBS object mapping. It gives Rust code a bridge-root dictionary, typed Rust payload mapping, nested read-back, and generated mapping support while keeping direct OOP access available.

What Is Supported

The initial mapping layer supports:

- `Session::bridge_root()`
- `Session::bridge_root_named(name)`
- `BridgeRoot::put`
- `BridgeRoot::put_mapped`
- `BridgeRoot::get_oop`
- `BridgeRoot::get_value`
- `BridgeRoot::get_string`
- `BridgeRoot::get_smallint`
- `BridgeRoot::get_bool`
- `BridgeRoot::get_dictionary`
- `BridgeRoot::get_mapped`
- `BridgeRoot::remove`
- `BridgeRoot::commit`
- `BridgeRoot::commit_with_retry`
- `BridgeRoot::transaction`
- `BridgeRoot::keys`
- `BridgeDictionary::at_oop`
- `BridgeDictionary::at_value`
- `BridgeDictionary::at_string`
- `BridgeDictionary::at_smallint`
- `BridgeDictionary::at_bool`
- `BridgeDictionary::at_dictionary`
- `BridgeDictionary::at_mapped`
- `BridgeDictionary::at_vec`
- `BridgeDictionary::keys`
- `BridgeKey`
- `BridgeKeyType::String`
- `BridgeKeyType::Symbol`
- `BridgeMapped`
- `#[derive(BridgeMapped)]`

- `BridgeFieldRead`
- `BridgeFieldWrite`
- `BridgeValue::Nil`
- `BridgeValue::Bool`
- `BridgeValue::SmallInt`
- `BridgeValue::String`
- `BridgeValue::Symbol`
- `BridgeValue::Oop`
- `BridgeValue::Dictionary`
- `BridgeValue::KeyedDictionary`
- `BridgeValue::Array`
- session-local OOP identity ids with `Session::identity_for_oop`

The default root is a `GemStone Dictionary` stored in `UserGlobals` under `#GemStoneRsBridgeRoot`.

Rust Example

```
use gemstone_rs::{BridgeValue, Config, Session};

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env())?;

    let payload = BridgeValue::dictionary([
        ("name".to_string(), BridgeValue::from("Tariq")),
        ("amount".to_string(), BridgeValue::from(100_i64)),
        ("currency".to_string(), BridgeValue::from("GBP")),
    ]);

    let mut bridge_root = session.bridge_root()?;
    let payload_oop = bridge_root.put("MyTestDict", payload)?;
    let stored = bridge_root.get_oop("MyTestDict"?);
    assert_eq!(payload_oop, stored);

    bridge_root.commit()?;
    Ok(())
}
```

Run the example:

```
cargo run -p gemstone-rs --example bridge_root_mapping
```

Expected output includes:

```
bridge root: GemStoneRsBridgeRoot
MyTestDict OOP: <number>
```

Typed Rust Struct Mapping

For application code, implement `BridgeMapped` on a plain Rust type. The trait keeps the mapping explicit and reviewable while removing repeated dictionary field reads from application code.

```
use gemstone_rs::{BridgeDictionary, BridgeMapped, BridgeValue, Config, Session};

#[derive(Debug, Eq, PartialEq)]
struct BookingDraft {
    name: String,
    amount: i64,
    currency: String,
}

impl BridgeMapped for BookingDraft {
    fn to_bridge_value(&self) -> BridgeValue {
        BridgeValue::dictionary([
            ("name".to_string(), BridgeValue::from(self.name.clone())),
            ("amount".to_string(), BridgeValue::from(self.amount)),
            ("currency".to_string(), BridgeValue::from(self.currency.clone())),
        ])
    }

    fn from_bridge_dictionary(dictionary: &mut BridgeDictionary<'_>) ->
gemstone_rs::Result<Self> {
        Ok(Self {
            name: dictionary.at_string("name")?,
            amount: dictionary.at_smallint("amount")?,
            currency: dictionary.at_string("currency")?,
        })
    }
}

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env())?;
    let mut bridge_root = session.bridge_root()?;

    let draft = BookingDraft {
        name: "Tariq".to_string(),
        amount: 100,
        currency: "GBP".to_string(),
    };

    bridge_root.put_mapped("MyTestDict", &draft)?;
    let loaded: BookingDraft = bridge_root.get_mapped("MyTestDict")?;
    assert_eq!(loaded, draft);

    bridge_root.commit()?;
```

```
    Ok(())  
}
```

The checked-in example uses this pattern:

```
cargo run -p gemstone-rs --example bridge_root_mapping
```

Expected output includes:

```
bridge root: GemStoneRsBridgeRoot  
MyTestDict OOP: <number>  
loaded payload: BookingDraft { name: "Tariq", amount: 100, currency: "GBP" }
```

Derive-Based Mapping

For normal Rust structs, prefer `#[derive(BridgeMapped)]`. The derive writes a `BridgeMapped` implementation that stores fields in a GemStone dictionary and reads them back into Rust.

```
use gemstone_rs::{BridgeMapped, Config, Session};  
  
#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]  
struct CustomerDraft {  
    #[bridge(key_type = "Symbol")]  
    name: String,  
}  
  
#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]  
struct BookingDraft {  
    #[bridge(key = "amount", key_type = "Symbol")]  
    amount: i64,  
    customer: CustomerDraft,  
    tags: Vec<String>,  
}  
  
fn main() -> gemstone_rs::Result<()> {  
    let mut session = Session::login(Config::from_env())?;  
    let mut bridge_root = session.bridge_root()?;  
  
    let draft = BookingDraft {  
        amount: 100,  
        customer: CustomerDraft {  
            name: "Tariq".to_string(),  
        },  
        tags: vec!["priority".to_string(), "demo".to_string()],  
    };  
  
    bridge_root.put_mapped("DerivedBookingDraft", &draft)?;  
    let loaded: BookingDraft = bridge_root.get_mapped("DerivedBookingDraft")?;
```



```

    assert_eq!(loaded, draft);

    bridge_root.commit()?;
    Ok(())
}

```

Run the checked-in live example:

```
cargo run -p gemstone-rs --example derive_mapping
```

Expected output includes:

```

derived mapped payload: BookingDraft { amount: 100, customer: CustomerDraft { name:
"Tariq" }, tags: ["priority", "demo"] }
bridge root identity: <number>

```

Nested Read-Back

Nested mapped structs and arrays round-trip through the same bridge dictionary format:

Rust field	GemStone storage	Read-back helper
String	String	BridgeFieldRead / at_string
i64	SmallInteger	BridgeFieldRead / at_smallint
bool	true / false	BridgeFieldRead / at_bool
Oop	raw object reference	BridgeFieldRead / at_oop
another BridgeMapped struct	nested Dictionary	BridgeDictionary::at_mapped
Vec<T>	Array	BridgeDictionary::at_vec

This lets a Rust payload keep normal nested shape:

```

#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct BookingDraft {
    customer: CustomerDraft,
    tags: Vec<String>,
}

```

When read-back fails, mapping errors include field context:

```
field amount expected GemStone value type SmallInt, got Oop 1234
field tags[2] expected GemStone value type String, got OOP 1234
```

Array read-back now reports the failing element index, which is important when generated mappings read back nested payloads from a live stone.

Key Policy

GemStone dictionaries often use either string keys or symbol keys. The mapping layer makes that explicit:

```
#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct BookingDraft {
    #[bridge(key = "amount", key_type = "Symbol")]
    amount: i64,
}
```

Manual code can use the same policy:

```
use gemstone_rs::BridgeKeyType;

bridge_root.put_with_key_type("BookingDraft", BridgeKeyType::Symbol, "stored"?;
let oop = bridge_root.get_oop_with_key_type("BookingDraft", BridgeKeyType::Symbol)?;
```

Use string keys when interoperating with JSON-like payloads and symbol keys when you want Smalltalk-style dictionary access.

Transactions and Identity

`BridgeRoot::transaction` commits on success and aborts on error:

```
bridge_root.transaction(|root| {
    root.put_mapped("BookingDraft", &draft)?;
    let loaded: BookingDraft = root.get_mapped("BookingDraft"?;
    assert_eq!(loaded, draft);
    Ok(())
})?;
```

For conflict-prone commits, use a bounded retry:

```
bridge_root.put_mapped("BookingDraft", &draft)?;  
bridge_root.commit_with_retry(2)?;
```

`Session` also tracks a session-local identity id for OOPs it sees:

```
let oop = bridge_root.get_oop("BookingDraft"?);  
let identity = bridge_root.identity_id();  
println!("bridge root identity={identity}, payload oop={}", oop.raw());
```

That is not transparent object persistence. It is a stable in-session cache key for wrappers, inspectors, and explorer views.

Comparison With MagLev/GBS

The MagLev branch example uses:

```
session bridgeRoot at: #MyTestDict put: payload.  
session commitTransactionOrSignalConflict
```

The closest current `gemstone-rs` shape is:

```
let mut bridge_root = session.bridge_root()?;  
bridge_root.put("MyTestDict", payload)?;  
bridge_root.commit()?;
```

For typed Rust structs:

```
bridge_root.put_mapped("MyTestDict", &draft)?;  
let loaded: BookingDraft = bridge_root.get_mapped("MyTestDict"?);
```

This is still explicit mapping, not transparent persistence. The benefit is that Rust teams can review every field mapping and still keep direct OOP access available when they need it.

Codegen Support

`gemstone-rs` codegen can emit `BridgeMapped` structs from config:

```
mapped = BookingDraft | doc=A typed Rust payload stored under BridgeRoot.  
field = BookingDraft.name | type=String | key=name  
field = BookingDraft.amount | type=SmallInt | key=amount | key_type=Symbol
```

```
field = BookingDraft.currency | type=String | key=currency
field = BookingDraft.tags | type=Vec<String> | key=tags
```

Generate a mapping config proposal from a live GemStone class:

```
gemstone-rs codegen discover-mapping gemstone-rs.codegen BookingDraft Object
```

For quick CLI inspection of the bridge root itself:

```
gemstone-rs bridge root
gemstone-rs bridge keys
gemstone-rs bridge get BookingDraft --symbol
gemstone-rs bridge inspect BookingDraft --symbol
gemstone-rs bridge sample-config BookingDraft
```

Use `--root OtherBridgeRoot` when you intentionally use a non-default root dictionary, and `--key-type String|Symbol` when you want the key policy to be spelled out in scripts.

`bridge keys` lists each root key with its key OOP, class OOP, `printString`, and session-local identity id.

Run the live discovery example:

```
cargo run -p gemstone-rs --example codegen_discover_mapping
```

Run the generated mapping example:

```
cargo run -p gemstone-rs --example generated_mapping_app
```

This is the recommended path for repeated mappings because the generated source is checked in and reviewed like any other Rust code.

Explorer and VS Code Support

The local explorer exposes BridgeRoot-oriented endpoints:

```
curl -s http://127.0.0.1:8787/api/bridge/root
curl -s http://127.0.0.1:8787/api/bridge/keys
curl -s 'http://127.0.0.1:8787/api/bridge/get?key=BookingDraft'
curl -s 'http://127.0.0.1:8787/api/bridge/put?key=WorkbenchDraft&value=hello'
curl -s 'http://127.0.0.1:8787/api/bridge/remove?key=WorkbenchDraft'
curl -s 'http://127.0.0.1:8787/api/bridge/mapping-config?mapped=BookingDraft'
```

```
curl -s 'http://127.0.0.1:8787/api/codegen/discover-mapping?
mapped=BookingDraft&class=Object'
```

The write endpoints require `gemstone-rs-explorer --allow-write`. Without that flag they return HTTP 403, matching the explorer's read-only default.

The VS Code workbench adds the same object-mapping workflow:

- GemStone RS: Generate Mapping Config
- GemStone RS: Preview BridgeRoot
- GemStone RS: List BridgeRoot Keys
- GemStone RS: Put BridgeRoot String
- GemStone RS: Remove BridgeRoot Key
- GemStone RS: Run Generated Mapping Example

The sidebar also shows these actions under `Codegen Config`.

Current Boundaries

The mapping layer is explicit, not transparent persistence. It does not yet make every GemStone object look like a native Rust object automatically. The current design is deliberately conservative:

- `Oop` remains visible and available.
- `Session` is still non-`Send` and non-`Sync`.
- writes happen only through explicit `put`, `put_mapped`, or generated code.
- the identity map is session-local and does not replace GemStone identity.

Rust Codegen

`gemstone-rs` codegen creates small Rust wrapper structs around GemStone OOPs. The goal is to move repeated selector strings into checked-in generated code while keeping the mapping reviewable.

Install

```
cargo install gemstone-rs-cli
gemstone-rs codegen --help
```

From a checkout:

```
cargo run -p gemstone-rs-cli -- codegen preview examples/codegen/gemstone-rs.codegen
```

Config Format

The config is intentionally line-oriented:

```
output = examples/codegen/generated/gemstone_wrappers.rs
class = Object
method = Object>>printString | return=String | doc=Return the receiver printString.
method = Object>>class
```

Use `Dictionary:ClassName` when a class should resolve from a specific dictionary:

```
class = UserGlobals:OkzBooking
method = UserGlobals:OkzBooking>>findById: | args=id | return=Oop | doc=Find a
        booking by id.
```

Class-side references use `class`:

```
class = UserGlobals:OkzBooking class
method = UserGlobals:OkzBooking class>>findById: | args=id | return=Oop
```

Method Metadata

Option	Example	Purpose
<code>args</code>	<code>args=id,user</code>	Names generated Rust function arguments.
<code>return</code>	<code>return=String</code>	Generates a typed return helper instead of <code>Value</code> .
<code>doc</code>	<code>doc=Find by id.</code>	Writes a Rust doc comment above the generated method.

When `args` is omitted, codegen now infers useful argument names from selector keywords. For example `at:put:` becomes `at, put`, and `withCustomer:amount:` becomes `customer, amount`. Provide explicit `args=...` when the selector keywords are not good Rust argument names.

Supported return types:

Config value	Rust return
Value	gemstone_rs::Value
String	String
SmallInt	i64
Bool	bool
Oop	gemstone_rs::Oop

Mapped Structs

Codegen can also emit typed `BridgeMapped` structs for values stored under `BridgeRoot` :

```
mapped = BookingDraft | doc=A typed Rust payload stored under BridgeRoot.
field = BookingDraft.name | type=String | key=name
field = BookingDraft.amount | type=SmallInt | key=amount | key_type=Symbol
field = BookingDraft.currency | type=String | key=currency
field = BookingDraft.tags | type=Vec<String> | key=tags
```

Supported field types are `String` , `SmallInt` , `Bool` , `Oop` , `Mapped<OtherStruct>` , and `Vec<T>` .

Field options:

Option	Example	Purpose
type	type=Vec<String>	Rust/GemStone field conversion.
key	key=amount	GemStone dictionary key.
key_type	key_type=Symbol	Use string keys or symbol keys explicitly.
doc	doc=Booking amount.	Writes a Rust doc comment above the field.

Commands

Create a starter config:

```
gemstone-rs codegen init gemstone-rs.codegen
```

Generate a config from a live stone:

```
gemstone-rs codegen discover gemstone-rs.codegen Object
gemstone-rs codegen discover gemstone-rs.codegen UserGlobals:OkzBooking
gemstone-rs codegen discover-mapping gemstone-rs.codegen BookingDraft Object
```

From a checkout:

```
cargo run -p gemstone-rs --example codegen_discover
cargo run -p gemstone-rs --example codegen_discover_mapping
```

Preview without writing:

```
gemstone-rs codegen preview gemstone-rs.codegen
```

Show a generated diff:

```
gemstone-rs codegen diff gemstone-rs.codegen
```

Check freshness in CI:

```
gemstone-rs codegen check gemstone-rs.codegen
```

Write generated wrappers:

```
gemstone-rs codegen generate gemstone-rs.codegen
```

Generated Shape

For:

```
method = Object>>printString | return=String | doc=Return the receiver printString.
```

The generated wrapper includes:

```
pub fn print_string(&mut self) -> Result<String> {
    let value = self.session.perform(self.oop, "printString", &[])?;
    match value {
        Value::String(value) => Ok(value),
        Value::Oop(oop) => self.session.fetch_string(oop),
        other => Err(Error::UnexpectedType {
```



```

        expected: "String",
        actual: format!("{other:?}"),
    }},
}

```

Generated files are meant to be checked in. That makes diffs, reviews, and editor indexing predictable.

Generated Wrapper App

The repository includes a small live example that imports the checked-in generated wrapper and calls `Object>>printString` on a small integer:

```
cargo run -p gemstone-rs --example generated_wrapper_app
```

Expected output:

```
generated wrapper printString: 7
```

The example uses the generated `Object::from_oop` constructor:

```

#[path = "../../examples/codegen/generated/gemstone_wrappers.rs"]
mod gemstone_wrappers;

use gemstone_rs::{Config, Session};

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env())?;
    let oop = session.smallint_oop(7);
    let mut object = gemstone_wrappers::Object::from_oop(&mut session, oop);
    println!("{}", object.print_string()?);
    Ok(())
}

```

Generated Object Mapping

The generated file can also include Rust data structs that implement `BridgeMapped`. Run:

```
cargo run -p gemstone-rs --example generated_mapping_app
```

Expected output:

```
generated mapped payload: BookingDraft { name: "Tariq", amount: 100, currency: "GBP",
tags: ["priority", "demo"] }
```

The config-driven mapping emits a struct like:

```
#[derive(Clone, Debug, Eq, PartialEq)]
pub struct BookingDraft {
    pub name: String,
    pub amount: i64,
    pub currency: String,
    pub tags: Vec<String>,
}
```

and an implementation of `BridgeMapped` so it works with:

```
bridge_root.put_mapped("GeneratedBookingDraft", &draft)?;
let loaded: BookingDraft = bridge_root.get_mapped("GeneratedBookingDraft")?;
```

VS Code Workflow

The `gemstone-rs Workbench` extension exposes the same flow in the GemStone RS sidebar:

- Discover from Live Stone
- Generate Mapping Config
- Preview BridgeRoot
- Run Generated Mapping Example
- Preview Wrappers
- Diff Generated Output
- Check Freshness
- Generate Wrappers
- Open Codegen Docs

`Generate Wrappers` shows the diff first and asks before writing when output would change.

Explorer Guide

`gemstone-rs-explorer` is a local-only HTTP explorer built on the Rust API. It is useful for proving the browser, inspect, eval, and codegen surfaces before embedding richer tooling in VS Code or another frontend.

Install and Run

```
cargo install gemstone-rs-explorer
gemstone-rs-explorer --port 8787
```

From a checkout:

```
cargo run -p gemstone-rs-explorer -- --port 8787
```

Open:

```
http://127.0.0.1:8787/
```

The home page is a small browser UI over the same JSON endpoints. It can:

- browse dictionaries, classes, protocols, methods, and source
- run doctor/status checks
- inspect BridgeRoot and list keys
- run codegen sample/discover/preview/diff/check/generate from an editable

config path

- load and save the selected codegen config file; saving is still gated by

`--allow-write`, accepts a POST body for larger configs, and validates the config before writing

- render generated wrapper source, generated mapping config, colored unified diff output, and a side-by-side diff in a dedicated detail pane

- remember the current browser fields locally across reloads
- put/remove BridgeRoot values with explicit string/symbol key policy and string/small-int/bool value policy when `--allow-write` is enabled

Screenshot:

gemstone-rs Explorer

Local-only GemStone/S explorer.

- [/api/config](#)
- [/api/status](#)
- [/api/browse/dictionaries](#)
- [/api/browse/classes?dictionary=UserGlobals](#)
- [/api/browse/protocols?class=Object](#)
- [/api/browse/methods?class=Object&protocol=- all -](#)
- [/api/browse/source?class=Object](#)
- [/api/codegen/sample](#)
- [/api/codegen/preview?config=examples/codegen/gemstone-rs.codegen](#)
- [/api/codegen/diff?config=examples/codegen/gemstone-rs.codegen](#)
- [/api/codegen/check?config=examples/codegen/gemstone-rs.codegen](#)
- [/api/inspect?oop=20](#)

read_only=true allow_eval=false

Safety Defaults

The explorer:

- binds to `127.0.0.1` by default
- rejects non-loopback hosts
- starts read-only
- requires `--allow-eval` before workspace evaluation
- requires `--allow-write` before codegen writes
- reads GemStone credentials from the same `GS_*` environment as the CLI

Do not expose it publicly without adding authentication and transport security.

Browse Endpoints

Run these from a second terminal:

```
curl -s http://127.0.0.1:8787/health
curl -s http://127.0.0.1:8787/api/config
curl -s http://127.0.0.1:8787/api/doctor
curl -s 'http://127.0.0.1:8787/api/doctor?live=1'
```

```
curl -s http://127.0.0.1:8787/api/status
curl -s http://127.0.0.1:8787/api/browse/dictionaries
curl -s 'http://127.0.0.1:8787/api/browse/classes?dictionary=UserGlobals'
curl -s 'http://127.0.0.1:8787/api/browse/protocols?class=Object&meta=0'
curl -s 'http://127.0.0.1:8787/api/browse/methods?class=Object&protocol=--%20all%20--&meta=0'
curl -s 'http://127.0.0.1:8787/api/browse/source?class=Object&selector=printString&meta=0'
curl -s 'http://127.0.0.1:8787/api/inspect?oop=20'
```

Expected shapes:

```
{"status":"ok"}
{"host":"127.0.0.1","port":8787,"readOnly":true,"allowEval":false}
{"success":true,"environment":{"GS_PASSWORD":{"status":"set","masked":true}}}
{"connected":true,"sessionId":12345,"needsCommit":false,"inTransaction":false}
{"success":true,"dictionaries":["UserGlobals"]}
```

Eval

Eval is disabled by default:

```
gemstone-rs-explorer --allow-eval
```

Then:

```
curl -s 'http://127.0.0.1:8787/api/eval?source=3%20%2B%204'
```

Expected:

```
{"type":"smallInt","value":7}
```

Codegen Endpoints

The home page includes a Codegen Workflow panel. Set `Config path`, optionally enter a mapped Rust type and GemStone class, then use:

- `Load Config` to read the selected config file into the editor
- `Save Config` to POST the editor contents, validate them, and write the file

when the explorer was started with `--allow-write`

- `Sample Config` to view the starter config format

- **Discover Mapping** to ask a live stone for a mapping config proposal
- **Preview** to inspect generated Rust wrappers without writing files
- **Diff** to compare generated output with the committed file; the detail pane

shows both the exact unified diff and a side-by-side view for review

- **Check** to fail when generated output is stale
- **Generate** to write wrappers when the explorer was started with

`--allow-write`

Read-only endpoints:

```
curl -s http://127.0.0.1:8787/api/codegen/sample
curl -s 'http://127.0.0.1:8787/api/codegen/config?config=examples/codegen/gemstone-rs.codegen'
curl -s 'http://127.0.0.1:8787/api/codegen/preview?config=examples/codegen/gemstone-rs.codegen'
curl -s 'http://127.0.0.1:8787/api/codegen/diff?config=examples/codegen/gemstone-rs.codegen'
curl -s 'http://127.0.0.1:8787/api/codegen/check?config=examples/codegen/gemstone-rs.codegen'
curl -s 'http://127.0.0.1:8787/api/codegen/discover-mapping?mapped=BookingDraft&class=Object'
```

Expected check result when generated output is current:

```
{"success":true,"exists":true,"upToDate":true}
```

Write-gated endpoint:

```
gemstone-rs-explorer --allow-write
```

```
curl -s 'http://127.0.0.1:8787/api/codegen/generate?config=examples/codegen/gemstone-rs.codegen'
curl -s -X POST \
  --data-binary @examples/codegen/gemstone-rs.codegen \
  'http://127.0.0.1:8787/api/codegen/config/save?config=examples/codegen/draft.codegen'
```

Expected:

```
{"success":true,"output":"examples/codegen/generated/gemstone_wrappers.rs","bytes":1234}
{"success":true,"config":"examples/codegen/draft.codegen","bytes":512}
```

BridgeRoot Endpoints

BridgeRoot inspection and mapping-config preview use the same live session configuration as the browser endpoints:

```
curl -s http://127.0.0.1:8787/api/bridge/root
curl -s http://127.0.0.1:8787/api/bridge/keys
curl -s 'http://127.0.0.1:8787/api/bridge/get?key=BookingDraft'
curl -s 'http://127.0.0.1:8787/api/bridge/get?key=BookingDraft&key_type=Symbol'
curl -s 'http://127.0.0.1:8787/api/bridge/mapping-config?mapped=BookingDraft'
```

The browser UI exposes **Bridge key type** and **Bridge value type** controls so you can explicitly test string-key, symbol-key, string-value, small-int-value, and bool-value mappings before encoding them in a reusable config file. It persists the selected key, value, class, selector, config, and mapping fields in browser local storage; use **Clear Saved Fields** to reset the page back to the documented defaults.

Writes are disabled unless the explorer is started with **--allow-write**:

```
gemstone-rs-explorer --allow-write
curl -s 'http://127.0.0.1:8787/api/bridge/put?key=WorkbenchDraft&value=hello'
curl -s 'http://127.0.0.1:8787/api/bridge/put?
key=WorkbenchCount&value=7&value_type=SmallInt'
curl -s 'http://127.0.0.1:8787/api/bridge/put?
key=WorkbenchApproved&value=true&value_type=Bool'
curl -s 'http://127.0.0.1:8787/api/bridge/remove?key=WorkbenchDraft'
```

Expected shapes:

```
{"success":true,"name":"GemStoneRsBridgeRoot","oop":1234,"identityId":1}
{"success":true,"root":"GemStoneRsBridgeRoot","keys":
[{"printString":"BookingDraft"}]}
{"oop":5678,"classOop":9012,"printString":"aDictionary(...)"
{"success":true,"config":"mapped = BookingDraft ..."}
{"success":true,"root":"GemStoneRsBridgeRoot","key":"WorkbenchDraft","keyType":"String",
"valueType":"String","oop":3456}
```

Product Direction

The explorer should remain a separate binary crate. The core **gemstone-rs** library stays focused on safe GemStone API access, while the explorer proves higher-level browsing and codegen workflows.

Good next steps:

- file-picker style config selection

- deeper VS Code webview integration

VS Code Workbench

`gemstone-rs Workbench` is the VS Code companion for the Rust CLI and local explorer. It keeps the CLI as the stable contract and adds a sidebar, output panel, generated-file previews, and codegen diff prompts.

Marketplace:

```
https://marketplace.visualstudio.com/items?itemName=unicompute.gemstone-rs-workbench
```

Setup

For installed CLI tools:

```
{
  "gemstoneRs.cliPath": "gemstone-rs",
  "gemstoneRs.explorerPath": "gemstone-rs-explorer",
  "gemstoneRs.codegenConfig": "gemstone-rs.codegen"
}
```

For a source checkout:

```
{
  "gemstoneRs.checkoutPath": "/path/to/gemstone-rs",
  "gemstoneRs.useCargo": true,
  "gemstoneRs.codegenConfig": "examples/codegen/gemstone-rs.codegen"
}
```

Use the same `GS_*` environment as the CLI.

Sidebar Tree

Open the GemStone RS activity bar item.

The sidebar contains:

- `Dictionaries`
- `Codegen Config`
- `Explorer`

Dictionaries expands live dictionaries, classes, protocols, and methods. Selecting a method opens its source in an untitled Smalltalk editor.

Codegen Config exposes:

- Discover from Live Stone
- Generate Mapping Config
- Preview BridgeRoot
- List BridgeRoot Keys
- Put BridgeRoot String
- Remove BridgeRoot Key
- Run Generated Mapping Example
- Preview Wrappers
- Diff Generated Output
- Check Freshness
- Generate Wrappers
- Open Codegen Docs

Explorer exposes:

- Doctor
- Verify Setup
- Eval Smalltalk
- Launch Explorer
- Open Explorer Webview

Command Palette

Commands:

- GemStone RS: Verify Setup
- GemStone RS: Doctor
- GemStone RS: Eval Smalltalk
- GemStone RS: Browse Dictionaries
- GemStone RS: Browse Classes
- GemStone RS: Codegen Init
- GemStone RS: Codegen Discover
- GemStone RS: Generate Mapping Config
- GemStone RS: Preview BridgeRoot
- GemStone RS: List BridgeRoot Keys
- GemStone RS: Put BridgeRoot String
- GemStone RS: Remove BridgeRoot Key
- GemStone RS: Run Generated Mapping Example
- GemStone RS: Codegen Preview
- GemStone RS: Codegen Diff
- GemStone RS: Codegen Check

- GemStone RS: Codegen Generate
- GemStone RS: Launch Explorer
- GemStone RS: Open Explorer Webview
- GemStone RS: Open Method Source
- GemStone RS: Open Codegen Docs

GemStone RS: Verify Setup and GemStone RS: Doctor both run the CLI `gemstone-rs doctor` report, so the workbench setup view and terminal setup use the same diagnostic path.

Codegen Workflow

1. Set `gemstoneRs.codegenConfig`.
2. Run GemStone RS: Codegen Discover or edit the config manually.
3. Run GemStone RS: Codegen Preview.
4. Run GemStone RS: Codegen Diff.
5. Run GemStone RS: Codegen Generate.

Codegen Generate runs the diff first. If output would change, it opens the diff and asks before writing.

Object Mapping Workflow

Use GemStone RS: Generate Mapping Config when you want the live stone to inspect a GemStone class and propose a BridgeMapped config. The command asks for:

- config path
- Rust struct name
- GemStone class reference, such as `Object` or `UserGlobals:OkzBooking`

Use GemStone RS: Preview BridgeRoot after starting the local explorer. It opens:

```
http://127.0.0.1:8787/api/bridge/root
```

Use GemStone RS: List BridgeRoot Keys for the same inspection path without opening a browser; it runs `gemstone-rs bridge keys` and writes the key OOPs, class OOPs, `printString` values, and identity ids to the output panel.

Use GemStone RS: Put BridgeRoot String when you want a quick live-write smoke test from VS Code. It asks for a key and string value, then runs:

```
gemstone-rs bridge put <key> <value> --type String
```

Use GemStone RS: Remove BridgeRoot Key to remove one BridgeRoot key after a confirmation prompt:

```
gemstone-rs bridge remove <key>
```

Use `GemStone RS: Run Generated Mapping Example` to run:

```
cargo run -p gemstone-rs --example generated_mapping_app
```

Embedded Explorer Webview

Use `GemStone RS: Launch Explorer` first. It starts `gemstone-rs-explorer` in a terminal and opens the browser. Then use `GemStone RS: Open Explorer Webview` to embed the local explorer inside VS Code.

The webview points at:

```
http://127.0.0.1:8787/
```

It is still the same loopback-only explorer process. If the explorer is not running, the webview will show a connection error and the output panel will show the URL to start.

The embedded page includes the same browse, BridgeRoot, and Codegen Workflow panels as the external browser view. That makes it the quickest place to preview generated wrappers, inspect unified and side-by-side diffs, run `codegen check`, and test BridgeRoot key/value policies without leaving VS Code. The explorer page can load and save the selected codegen config file, posting the editor contents as the request body; saves still require the explorer to run with `--allow-write`. It also remembers its current fields locally, so repeated codegen or BridgeRoot checks keep the same config path and class selection.

Develop the Extension

```
cd vscode-gemstone-rs-workbench
npm ci
npm run check
npm run test:smoke
npm run package -- --out gemstone-rs-workbench-0.3.0.vsix
```

From the repository root:

```
make vscode-package
```

Later Feature Work

The first embedded webview is available. Next work should make the webview more IDE-like: generate wrapper diffs before writing, edit codegen selections, inspect BridgeRoot payloads visually, and keep browser fallback behavior for users who prefer the external explorer.

Performance and Safety

`gemstone-rs` keeps the low-level GemStone/S GCI boundary explicit. The goal is to make Rust services and tooling fast without hiding session, transaction, or threading rules.

Dynamic GCI Loading

The GCI library is loaded at runtime. Configure it with one of:

```
export GS_LIB=/path/to/GemStone64Bit/lib
export GS_LIB_PATH=/path/to/GemStone64Bit/lib
export GEMSTONE=/path/to/GemStone64Bit
```

The loader should keep improving in these directions:

- report every searched path in `gemstone-rs doctor`
- detect architecture mismatches early
- avoid requiring GemStone headers at Rust build time
- keep all unsafe C ABI calls isolated in `gemstone-gci`

Session Threading

`Session` is deliberately non-`Send` and non-`Sync`. Treat a session as owned by the thread that logged it in.

For web servers:

- use `spawn_blocking` for synchronous GemStone calls from async Rust
- prefer one session per request until a proven session pool exists
- keep transaction boundaries explicit
- do not share a live session between async tasks

Benchmark Smoke Test

Use the lightweight benchmark smoke script for a coarse local sanity check:

```
ITERATIONS=20 scripts/benchmark_smoke.sh "3 + 4"
```

This measures CLI startup, login, eval, and logout together. It is not a microbenchmark. A future benchmark suite should separate:

- GCI library load time
- login/logout time
- eval/perform latency
- string and array marshalling
- BridgeRoot read/write latency
- codegen wrapper overhead

Rust vs Python

Compare `gemstone-rs` and `gemstone-py` only with equivalent workloads:

- same stone
- same host
- same GemStone user
- same expression or selector
- warm and cold runs reported separately

The first useful comparison is:

```
Rust: cargo run -p gemstone-rs-cli -- eval "3 + 4"
Python: python -m gemstone_py.cli eval "3 + 4"
```

After that, compare generated-wrapper calls and BridgeRoot mapping operations.

Safety Checklist

- Keep `Session` non-`Send` and non-`Sync`.
- Keep `unsafe` in `gemstone-gci`.
- Prefer typed `Value` / `Oop` conversions over raw integer handling.
- Commit or abort explicitly after writes.
- Bind local tools to `127.0.0.1` by default.
- Keep explorer eval/write operations opt-in.

Shared Core Integration

The long-term direction is:

```
gemstone-gci
  Low-level dynamic libgci loader and raw GCI calls.

gemstone-rs
  Safe Rust API: Config, Session, Oop, Value, browser, codegen, BridgeRoot.

gemstone-py-native
  Thin PyO3 wrapper around the Rust core.

gemstone-py
  Python API, async/web adapters, examples, docs, and compatibility layer.
```

This keeps Python and Rust from growing separate native bridges.

What Should Move Into Rust

- dynamic GCI library loading
- OOP constants and conversions
- session login/logout
- eval, execute, perform
- global get/put
- string, symbol, array, and dictionary marshalling
- transaction commit/abort
- browser primitives
- codegen discovery primitives

What Should Stay Python-Specific

- Pythonic `Session` and async wrappers
- FastAPI, Litestar, Django examples
- Python package extras such as `gemstone-py[fast]`
- backward-compatible return behavior
- Python docs and notebooks

PyO3 Wrapper Shape

`gemstone-py-native` should become a small adapter:

```
Python call -> PyO3 wrapper -> gemstone-rs Session -> gemstone-gci -> libgcirpc
```

The wrapper should expose stable operations first:

- `login(config)`
- `eval(source)`
- `execute(source)`
- `perform(oop, selector, args)`
- `commit()`
- `abort()`
- `logout()`

Only after that should it expose higher-level browser, codegen, and BridgeRoot operations.

Migration Plan

1. Keep `gemstone-rs` independent and publishable.
2. Add a small PyO3 crate that depends on `gemstone-rs`.
3. Replace duplicated native loading code in `gemstone-py-native`.
4. Run the existing `gemstone-py` live tests through the Rust-backed native path.
5. Keep pure Python fallback behavior available.

The main design rule: Rust owns the native bridge; Python owns Python ergonomics.

Release Checklist

Use this checklist for coordinated crate, VSIX, and GitHub releases.

Before Release

- Update crate versions in `Cargo.toml` files.
- Update `vscode-gemstone-rs-workbench/package.json`.
- Regenerate codegen examples:

```
cargo run -p gemstone-rs-cli -- codegen generate examples/codegen/gemstone-rs.codegen
```

- Run local verification:

```
make verify
make vscode-package
python3 docs/build_pdf_docs.py
```

Or use the dry-run release wrapper:

```
DRY_RUN=1 scripts/release_all.sh 0.2.0
```

`make verify` checks that PDF generation completes and produces non-empty PDF files. The release workflow rebuilds and attaches fresh PDFs for the target runner because WeasyPrint output can differ byte-for-byte across platforms. The VSIX filename uses `vscode-gemstone-rs-workbench/package.json`; the crate release tag still uses the workflow `version` input.

Publish

Set GitHub secrets once:

```
gh secret set CARGO_REGISTRY_TOKEN
gh secret set VSCE_PAT
```

Run the release workflow:

```
gh workflow run release.yml \
  --ref main \
  -f version=0.2.0 \
  -f publish-crates=true \
  -f publish-vsix=true \
  -f create-github-release=true \
  -f dry-run=false
```

The workflow publishes crates in dependency order:

1. `gemstone-gci`
2. `gemstone-rs-macros`
3. `gemstone-rs`
4. `gemstone-rs-cli`
5. `gemstone-rs-explorer`

It also packages/publishes the VSIX and can create the GitHub release with the VSIX attached.

Manual crate publishing remains available:


```
scripts/publish_crates.sh
```

For a dry run:

```
DRY_RUN=1 scripts/publish_crates.sh
```

Manual end-to-end publishing remains available through the release wrapper:

```
PUBLISH_CRATES=1 PUBLISH_VSIX=1 CREATE_GITHUB_RELEASE=1 VERIFY_PUBLIC=1 DRY_RUN=0  
scripts/release_all.sh 0.2.0
```

Dry-run mode checks each crate's publish file list locally. The real publish path still uses `cargo publish` in dependency order so crates.io resolves each new dependency after it has been published. If a workflow is rerun after a partial publish, already-published crate versions are skipped and the remaining crates continue.

Post-Release Verification

```
scripts/publish_verify.sh 0.2.0
```

The script checks:

- crates.io package/version JSON for all crates
- `cargo install gemstone-rs-cli`
- `cargo install gemstone-rs-explorer`
- `gemstone-rs --help`
- `gemstone-rs-explorer --help`
- VS Code Marketplace version matches `package.json`
- GitHub Release `v<version>` exists
- GitHub Release assets include the VSIX and `SHA256SUMS`

To skip the GitHub Release asset check during early testing:

```
VERIFY_GITHUB_RELEASE=0 scripts/publish_verify.sh 0.2.0
```

Optional Live Smoke

Run the manual GitHub Actions live job with GemStone secrets configured, or run locally:

```
GS_RUN_LIVE_RUST=1 cargo test -p gemstone-rs live_ -- --test-threads=1
```

The live smoke coverage includes login/logout, `3 + 4 == 7`, global put/get, string round-trip, `perform`, commit, abort, browser lookup of `Object`, and generated wrapper `printString`.

The `--test-threads=1` flag is intentional. GemStone GCI has process-global session state, so live tests should not be run concurrently.

This PDF was generated locally from docs/. If you edit the Markdown sources, rerun `python docs/build_pdf_docs.py`.